

Resolucion del Practico 5

Testing de Software 2024

Índice

Introduccion	3
1. Ejercicio 1: Cobertura logica sobre cinco predicados	3
1.1. Predicado $p = a \vee (b \wedge c)$	3
1.2. Predicado $p = a \rightarrow (b \rightarrow c)$	5
1.3. Predicado $p = (a \vee b) \wedge (c \vee d)$	5
1.4. Predicado $p = a \oplus b$	7
1.5. Predicado $p = a \vee b \vee (c \wedge d)$	7
2. Ejercicio 2: <code>checkIt</code> y <code>checkItExpand</code>	9
2.1.Codigo de partida	9
2.2. (a) Transformacion a <code>checkItExpand</code> e instrumentacion	9
2.3. (b) Suites T1 (CACC) y T2 (Edge Coverage). ¿T2 satisface CACC?	10
2.4. (c) Implementacion JUnit de T1 y T2	11
3. Ejercicio 3: <code>Thermostat.turnHeaterOn</code>	12
3.1. Predicado bajo analisis	12
3.2. Diseno comun de los tests	12
3.3. (a) CC pero no PC	13
3.4. (b) PC pero no CC	13
3.5. (c) CACC	13
3.6. Implementaciones JUnit	14
4. Ejercicio 4: <code>TriTyp.triang</code>	14
4.1. Predicados identificados	14
4.2. Casos de prueba	15
4.3. Pares CACC por predicado	15
4.4. Implementacion JUnit	16

Introduccion

Este documento contiene la resolucion del *Practico 5* de Testing de Software, centrado en **testing basado en expresiones logicas**. Los predicados que aparecen en el codigo (decisiones de `if`, `while`, etc.), en especificaciones, en maquinas de estados o en requisitos, definen un espacio de combinaciones de valores de verdad mucho mas grande del que es razonable probar. Los criterios de cobertura logica seleccionan, de manera sistematica, un subconjunto significativo de esas combinaciones.

A lo largo de la resolucion se usan los siguientes criterios:

- **PC** (Predicate Coverage): cada predicado p se evalua a *true* y a *false*.
- **CC** (Clause Coverage): cada clausula c se evalua a *true* y a *false*.
- **CACC** (Correlated Active Clause Coverage): para cada clausula mayor c_i existe un par de tests donde c_i *determina* el predicado y, en ese par, p cambia de valor.
- **RACC** (Restricted Active Clause Coverage): igual que CACC, pero ademas las clausulas menores deben tener *exactamente los mismos valores* en ambos tests del par.

Determinacion de una clausula. Una clausula c_i determina a un predicado p cuando, fijando las demas clausulas (las *menores*), el valor de p es el mismo que el valor de c_i . Operativamente, c_i determina p si al cambiar c_i y dejar fijas las demas, p cambia.

Convenciones. En las tablas, las filas se numeran en orden lexicografico con T antes que F . Cuando se enumeran pares CACC/RACC, se listan como $(\text{fila}_i, \text{fila}_j)$.

1. Ejercicio 1: Cobertura logica sobre cinco predicados

Para cada uno de los predicados se identifican sus clausulas, se construye la tabla de verdad y se analiza cada criterio: CC, PC, CACC y RACC.

1.1. Predicado $p = a \vee (b \wedge c)$

(a) **Clausulas.** Las clausulas son a , b y c .

(b) **Tabla de verdad.**

Fila	a	b	c	p
1	T	T	T	T
2	T	T	F	T
3	T	F	T	T
4	T	F	F	T
5	F	T	T	T
6	F	T	F	F
7	F	F	T	F
8	F	F	F	F

(c) CC pero no PC. Si es posible. Si la suite mantiene p constante pero cada clausula cambia, se logra CC sin PC.

Suite: {fila 4, fila 5} = $\{(T, F, F), (F, T, T)\}$. Cada clausula toma T y F , y en ambos casos $p = T$. Se cumple CC y no se cumple PC.

(d) PC pero no CC. Si es posible.

Suite: {fila 1, fila 6} = $\{(T, T, T), (F, T, F)\}$. Aqui p toma T y F (PC), pero la clausula b se queda fija en T , asi que CC falla.

(e) Pares CACC. Para cada clausula mayor se buscan pares en los que, fijando un valor de las menores que permita la determinacion, el predicado cambie:

- Mayor a : las menores deben hacer $b \wedge c = F$. Pares cruzados entre $\{2, 3, 4\}$ y $\{6, 7, 8\}$, es decir 9 pares en total: $(2, 6), (2, 7), (2, 8), (3, 6), (3, 7), (3, 8), (4, 6), (4, 7), (4, 8)$.
- Mayor b : para que b determine $(b \wedge c)$ debe ser $c = T$; ademas se necesita $a = F$ para que a no fuerce p . Par: $(5, 7)$.
- Mayor c : simetricamente, $b = T$ y $a = F$. Par: $(5, 6)$.

(f) Pares RACC. RACC exige que las clausulas menores tengan los *mismos* valores en ambos tests del par:

- Mayor a : parejas con (b, c) iguales. $(2, 6)$ con $(b, c) = (T, F)$; $(3, 7)$ con (F, T) ; $(4, 8)$ con (F, F) .
- Mayor b : $(5, 7)$, con $(a, c) = (F, T)$.
- Mayor c : $(5, 6)$, con $(a, b) = (F, T)$.

1.2. Predicado $p = a \rightarrow (b \rightarrow c)$

(a) **Clausulas.** a, b, c . Equivalentemente $p \equiv \neg a \vee \neg b \vee c$.

(b) **Tabla de verdad.**

Fila	a	b	c	p
1	T	T	T	T
2	T	T	F	F
3	T	F	T	T
4	T	F	F	T
5	F	T	T	T
6	F	T	F	T
7	F	F	T	T
8	F	F	F	T

(c) **CC pero no PC.** Si es posible. El predicado solo es F en la fila 2; si se evita esa fila, p queda en T y aun se puede tener CC.

Suite: {fila 1, fila 8} = {(T, T, T), (F, F, F)}. Cada clausula toma T y F , p es T en ambos casos. CC se cumple, PC no.

(d) **PC pero no CC.** Si es posible.

Suite: {fila 1, fila 2} = {(T, T, T), (T, T, F)}. p toma ambos valores; a y b quedan fijos en T .

(e) **Pares CACC.**

- Mayor a : para que a determine p , el subpredicado $(b \rightarrow c)$ debe ser F , o sea $b = T, c = F$. Par: (2, 6).
- Mayor b : b determina cuando $a = T$ (sino p es T siempre) y $c = F$. Par: (2, 4).
- Mayor c : c determina cuando $a = T$ y $b = T$. Par: (1, 2).

(f) **Pares RACC.** Como las menores quedan fijadas a un unico valor, los pares CACC ya son RACC: (2, 6) para a , (2, 4) para b , (1, 2) para c .

1.3. Predicado $p = (a \vee b) \wedge (c \vee d)$

(a) **Clausulas.** a, b, c, d .

(b) Tabla de verdad.

Fila	a	b	c	d	p
1	T	T	T	T	T
2	T	T	T	F	T
3	T	T	F	T	T
4	T	T	F	F	F
5	T	F	T	T	T
6	T	F	T	F	T
7	T	F	F	T	T
8	T	F	F	F	F
9	F	T	T	T	T
10	F	T	T	F	T
11	F	T	F	T	T
12	F	T	F	F	F
13	F	F	T	T	F
14	F	F	T	F	F
15	F	F	F	T	F
16	F	F	F	F	F

(c) CC pero no PC. Si es posible.

Suite: {fila 4, fila 13} = $\{(T, T, F, F), (F, F, T, T)\}$. Cada clausula cambia, pero $p = F$ en ambos casos.

(d) PC pero no CC. Si es posible.

Suite: {fila 1, fila 4} = $\{(T, T, T, T), (T, T, F, F)\}$. p toma T y F pero a y b quedan fijas en T .

(e) Pares CACC.

- Mayor a : las menores deben hacer $b = F$ y $(c \vee d) = T$. Filas con $a = T$ que cumplen esas condiciones: 5, 6, 7. Filas con $a = F$ que las cumplen: 13, 14, 15. Pares: cualquier combinacion cruzada (9 pares).
- Mayor b : simetrica a a (con $a = F$, $(c \vee d) = T$): filas 9, 10, 11 frente a 13, 14, 15. 9 pares en total.
- Mayor c : necesita $d = F$ (sino $c \vee d = T$ siempre) y $(a \vee b) = T$. Filas con $c = T$ que cumplen $d = F$ y $(a \vee b) = T$: 2, 6, 10. Filas con $c = F$ en las mismas condiciones: 4, 8, 12. 9 pares.

- Mayor d : por simetria con c , $c = F$ y $(a \vee b) = T$. Filas con $d = T$: 3, 7, 11; filas con $d = F$: 4, 8, 12. 9 pares.

(f) Pares RACC. Se pide que las menores valgan lo mismo en cada par:

- Mayor a (fijando (b, c, d)): (5, 13) con (F, T, T) ; (6, 14) con (F, T, F) ; (7, 15) con (F, F, T) .
- Mayor b (fijando (a, c, d)): (9, 13), (10, 14), (11, 15).
- Mayor c (fijando (a, b, d)): (2, 4), (6, 8), (10, 12).
- Mayor d (fijando (a, b, c)): (3, 4), (7, 8), (11, 12).

1.4. Predicado $p = a \oplus b$

(a) Clausulas. a, b .

(b) Tabla de verdad.

Fila	a	b	p
1	T	T	F
2	T	F	T
3	F	T	T
4	F	F	F

(c) CC pero no PC. Si es posible.

Suite: {fila 1, fila 4} = $\{(T, T), (F, F)\}$. $p = F$ en ambos casos, pero ambas clausulas toman T y F .

(d) PC pero no CC. Si es posible.

Suite: {fila 1, fila 2} = $\{(T, T), (T, F)\}$. p toma ambos valores, pero a no toma F .

(e) y (f) Pares CACC y RACC. Para XOR, cada clausula siempre determina al predicado (cambiar una variable siempre cambia p). Ademas, fijando el valor de la otra clausula los pares son unicos, asi que CACC y RACC coinciden:

- Mayor a : (1, 3) con $b = T$; (2, 4) con $b = F$.
- Mayor b : (1, 2) con $a = T$; (3, 4) con $a = F$.

1.5. Predicado $p = a \vee b \vee (c \wedge d)$

(a) Clausulas. a, b, c, d .

(b) Tabla de verdad.

Fila	a	b	c	d	p
1	T	T	T	T	T
2	T	T	T	F	T
3	T	T	F	T	T
4	T	T	F	F	T
5	T	F	T	T	T
6	T	F	T	F	T
7	T	F	F	T	T
8	T	F	F	F	T
9	F	T	T	T	T
10	F	T	T	F	T
11	F	T	F	T	T
12	F	T	F	F	T
13	F	F	T	T	T
14	F	F	T	F	F
15	F	F	F	T	F
16	F	F	F	F	F

(c) CC pero no PC. Si es posible.

Suite: {fila 4, fila 13} = {(T, T, F, F), (F, F, T, T)}. Cada clausula cambia y $p = T$ en ambos casos.

(d) PC pero no CC. Si es posible.

Suite: {fila 1, fila 14} = {(T, T, T, T), (F, F, T, F)}. p toma ambos valores; c queda fija en T .

(e) Pares CACC.

- Mayor a : necesita $b = F$ y $(c \wedge d) = F$. Filas con $a = T$: 6, 7, 8; filas con $a = F$: 14, 15, 16. 9 pares cruzados.
- Mayor b : simetrica a a (con $a = F$ y $(c \wedge d) = F$): filas 10, 11, 12 frente a 14, 15, 16. 9 pares.
- Mayor c : c determina $(c \wedge d)$ cuando $d = T$. Ademas, para que $(c \wedge d)$ determine p , se necesita $a = F$ y $b = F$. Par: (13, 15).
- Mayor d : simetrico a c , con $c = T$, $a = F$, $b = F$. Par: (13, 14).

(f) **Pares RACC.** Fijando las menores en cada par:

- Mayor a (fijando (b, c, d)): $(6, 14)$ con (F, T, F) ; $(7, 15)$ con (F, F, T) ; $(8, 16)$ con (F, F, F) .
- Mayor b (fijando (a, c, d)): $(10, 14)$, $(11, 15)$, $(12, 16)$.
- Mayor c : $(13, 15)$.
- Mayor d : $(13, 14)$.

2. Ejercicio 2: checkIt y checkItExpand

2.1. Codigo de partida

```
public static void checkIt (boolean a, boolean b, boolean c) {  
    if (a && (b || c))  
        System.out.println ("P is true");  
    else  
        System.out.println ("P isn't true");  
}
```

El predicado bajo analisis es $p = a \wedge (b \vee c)$.

2.2. (a) Transformacion a checkItExpand e instrumentacion

Se reescribe el codigo para que *cada if use una sola variable booleana*. Una forma estandar es descomponer el if en if/else anidados:

```
public static ExecutionTrace checkItExpand(boolean a, boolean b,  
boolean c) {  
    nodes.add("START"); nodes.add("A");  
    if (a) {  
        nodes.add("B");  
        if (b) {  
            nodes.add("TRUE"); result = true;  
        } else {  
            nodes.add("C");  
            if (c) { nodes.add("TRUE"); result = true; }  
            else { nodes.add("FALSE"); result = false; }  
        }  
    } else {  
        nodes.add("FALSE"); result = false;  
    }  
    nodes.add("END");  
}
```

```

    return new ExecutionTrace(result, nodes);
}

```

Instrumentacion. `checkItExpand` devuelve un objeto `ExecutionTrace` que contiene:

- la lista de *nodos visitados* en orden;
- las *aristas atravesadas*, derivadas como pares consecutivos $N_i \rightarrow N_{i+1}$.

Los nodos del CFG son $\{\mathbf{START}, \mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{TRUE}, \mathbf{FALSE}, \mathbf{END}\}$ y las aristas posibles:

$\mathbf{START} \rightarrow \mathbf{A}$, $\mathbf{A} \rightarrow \mathbf{B}$, $\mathbf{A} \rightarrow \mathbf{FALSE}$, $\mathbf{B} \rightarrow \mathbf{TRUE}$, $\mathbf{B} \rightarrow \mathbf{C}$, $\mathbf{C} \rightarrow \mathbf{TRUE}$, $\mathbf{C} \rightarrow \mathbf{FALSE}$, $\mathbf{TRUE} \rightarrow \mathbf{END}$,

La instrumentacion permite verificar cobertura de aristas directamente desde los tests, sin depender de una herramienta externa.

2.3. (b) Suites T1 (CACC) y T2 (Edge Coverage). ¿T2 satisface CACC?

Suite T1 (CACC sobre `checkIt`).

$$T_1 = \{t_1 = (T, T, F), t_2 = (F, T, F), t_3 = (T, F, F), t_4 = (T, F, T)\}.$$

Justificacion por clausula mayor:

- Mayor a : $(t_1, t_2) = ((T, T, F), (F, T, F))$. Las menores $b \vee c$ valen T en ambos casos; p cambia de T a F .
- Mayor b : $(t_1, t_3) = ((T, T, F), (T, F, F))$. $a = T$ y $c = F$ fijos; p cambia de T a F .
- Mayor c : $(t_4, t_3) = ((T, F, T), (T, F, F))$. $a = T$ y $b = F$ fijos; p cambia de T a F .

Suite T2 (Edge Coverage sobre `checkItExpand`).

$$T_2 = \{e_1 = (F, F, F), e_2 = (T, T, F), e_3 = (T, F, T), e_4 = (T, F, F)\}.$$

Caminos:

- e_1 : $\mathbf{START} \rightarrow \mathbf{A} \rightarrow \mathbf{FALSE} \rightarrow \mathbf{END}$.
- e_2 : $\mathbf{START} \rightarrow \mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{TRUE} \rightarrow \mathbf{END}$.
- e_3 : $\mathbf{START} \rightarrow \mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{C} \rightarrow \mathbf{TRUE} \rightarrow \mathbf{END}$.

- e_4 : $\text{START} \rightarrow A \rightarrow B \rightarrow C \rightarrow \text{FALSE} \rightarrow \text{END}$.

La union cubre las 9 aristas. **T2** satisface **Edge Coverage**.

¿T2 satisface CACC sobre `checkIt`? No.

Para que a sea la clausula mayor en `checkIt`, hace falta un par donde $b \vee c = T$ y a cambie entre los dos tests. En T_2 los casos con $b \vee c = T$ son e_2 y e_3 , ambos con $a = T$. No hay ningun caso con $a = F$ y $b \vee c = T$. Por lo tanto, el par CACC para mayor a no existe en T_2 .

Edge Coverage sobre el CFG expandido y CACC sobre el predicado original son criterios distintos: la expansion estructural conserva la semantica del programa pero pierde la nocion de clausula activa, justamente porque cada `if` solo refiere a una variable.

2.4. (c) Implementacion JUnit de T1 y T2

T1 (CACC). La suite `CheckItT1CaccTest` verifica, para cada clausula mayor, que el par correspondiente produce un cambio en el predicado:

```
@Test public void caccMajorA() {
    assertTrue (CheckIt.checkIt(true, true, false));
    assertFalse(CheckIt.checkIt(false, true, false));
}
@Test public void caccMajorB() {
    assertTrue (CheckIt.checkIt(true, true, false));
    assertFalse(CheckIt.checkIt(true, false, false));
}
@Test public void caccMajorC() {
    assertTrue (CheckIt.checkIt(true, false, true));
    assertFalse(CheckIt.checkIt(true, false, false));
}
```

T2 (Edge Coverage). La suite `CheckItT2EdgeCoverageTest` ejecuta `checkItExpand` sobre los cuatro casos, junta las aristas y compara contra el conjunto esperado:

```
Set<String> covered = new LinkedHashSet<>();
covered.addAll(CheckIt.checkItExpand(false, false, false).
    getEdges());
covered.addAll(CheckIt.checkItExpand(true, true, false).
    getEdges());
covered.addAll(CheckIt.checkItExpand(true, false, true ).
    getEdges());
```

```
covered.addAll(CheckIt.checkItExpand(true, false, false).
    getEdges());

assertEquals(EXPECTED_EDGES, covered);
```

Un segundo test verifica explícitamente que esa suite *no* contiene ningún par CACC para *a*, dejando documentado el resultado del inciso (b).

Resultado de ejecución. Desde `tp5/assignment-5-rodeghiero`:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
-Djacoco.skip=true test
```

reporta BUILD SUCCESS con todos los tests pasando.

3. Ejercicio 3: Thermostat.turnHeaterOn

3.1. Predicado bajo analisis

La decision principal del metodo es

$$((curTemp < dTemp - thresholdDiff) \vee (override \wedge curTemp < overTemp - thresholdDiff)) \wedge (timeSinceLastRun > minLag)$$

que se reescribe en forma abstracta como

$$p = ((a \vee (b \wedge c)) \wedge d),$$

con las clausulas

- *a*: `curTemp < dTemp - thresholdDiff`,
- *b*: `override`,
- *c*: `curTemp < overTemp - thresholdDiff`,
- *d*: `timeSinceLastRun > minLag`.

3.2. Diseno comun de los tests

Para que cada test controle los valores de las cuatro clausulas, se fija: `Period.MORNING`, `DayType.WEEKDAY`, `setSetting(MORNING, WEEKDAY, 69)`, `thresholdDiff = 5` y `minLag = 10`. Variando `curTemp`, `override`, `overTemp` y `timeSinceLastRun` se elige el valor de cada clausula. La logica de armado esta en `ThermostatCoverageSupport.runCase(a,b,c,d)`,

que ademas devuelve los valores *reales* de cada clausula evaluados sobre el objeto para verificar que el setup coincide con la intencion del test.

3.3. (a) CC pero no PC

Suite propuesta:

- $t_1 = (a = T, b = F, c = F, d = F) \Rightarrow p = (T \vee F) \wedge F = F.$
- $t_2 = (a = F, b = T, c = F, d = T) \Rightarrow p = (F \vee F) \wedge T = F.$
- $t_3 = (a = F, b = F, c = T, d = T) \Rightarrow p = (F \vee F) \wedge T = F.$

Justificacion.

- Cada clausula toma T y F al menos una vez:
 - a : T en t_1 ; F en t_2, t_3 .
 - b : T en t_2 ; F en t_1, t_3 .
 - c : T en t_3 ; F en t_1, t_2 .
 - d : T en t_2, t_3 ; F en t_1 .

Por lo tanto **CC se cumple**.

- p vale F en los tres casos, asi que **PC no se cumple**.

3.4. (b) PC pero no CC

Suite propuesta:

- $u_1 = (a = T, b = T, c = T, d = T) \Rightarrow p = T.$
- $u_2 = (a = T, b = T, c = T, d = F) \Rightarrow p = F.$

p toma T y F , asi que PC se cumple. Sin embargo a , b y c no toman el valor F en ninguno de los dos tests; CC no se cumple.

3.5. (c) CACC

Se construye un par por cada clausula mayor.

Mayor a . Necesita $b \wedge c = F$ (sino el subpredicado $(a \vee (b \wedge c))$ no esta determinado por a) y $d = T$ (sino el $\wedge$ exterior anula a a).

$$(T, F, F, T) \Rightarrow p = T, \quad (F, F, F, T) \Rightarrow p = F.$$

Mayor b. Para que b determine a $b \wedge c$ debe ser $c = T$; ademàs $a = F$ y $d = T$.

$$(F, T, T, T) \Rightarrow p = T, \quad (F, F, T, T) \Rightarrow p = F.$$

Mayor c. Simetrica a b : $b = T$, $a = F$, $d = T$.

$$(F, T, T, T) \Rightarrow p = T, \quad (F, T, F, T) \Rightarrow p = F.$$

Mayor d. Necesita $(a \vee (b \wedge c)) = T$, lo cual se logra con $a = T$.

$$(T, F, F, T) \Rightarrow p = T, \quad (T, F, F, F) \Rightarrow p = F.$$

Reuniendo los pares y eliminando duplicados, la suite CACC contiene 6 tests distintos.

3.6. Implementaciones JUnit

Los tres incisos se materializan como tres clases en `src/test/java/assignment5_exercises/thermost`.

- `ThermostatExercise3ClauseCoverageTest`: verifica el item (a).
- `ThermostatExercise3PredicateCoverageTest`: verifica el item (b).
- `ThermostatExercise3CaccTest`: verifica el item (c) con cuatro pares CACC.

Las tres clases comparten `ThermostatCoverageSupport`, que ademàs chequea que el valor realmente devuelto por `turnHeaterOn` coincida con la evaluacion teorica del predicado. La suite completa pasa con `mvn test`.

4. Ejercicio 4: `TriTyp.triang`

4.1. Predicados identificados

`triang(s1,s2,s3)` contiene varios predicados que se evaluan en cascada. Los identificamos así:

1. $P_1 = (s_1 \leq 0) \vee (s_2 \leq 0) \vee (s_3 \leq 0)$, con clausulas c_1, c_2, c_3 .
2. $P_2 = (s_1 = s_2)$.
3. $P_3 = (s_1 = s_3)$.
4. $P_4 = (s_2 = s_3)$.
5. $P_5 = (triOut = 0)$.

6. $P_6 = (s_1 + s_2 \leq s_3) \vee (s_2 + s_3 \leq s_1) \vee (s_1 + s_3 \leq s_2)$, clausulas d_1, d_2, d_3 .
7. $P_7 = (triOut > 3)$.
8. $P_8 = (triOut = 1) \wedge (s_1 + s_2 > s_3)$, clausulas e_1, e_2 .
9. $P_9 = (triOut = 2) \wedge (s_1 + s_3 > s_2)$, clausulas f_1, f_2 .
10. $P_{10} = (triOut = 3) \wedge (s_2 + s_3 > s_1)$, clausulas g_1, g_2 .

Para CACC distinguimos:

- Predicados con varias clausulas ($P_1, P_6, P_8, P_9, P_{10}$): un par por cada clausula mayor.
- Predicados de una sola clausula (P_2, P_3, P_4, P_5, P_7): basta con que cada uno tome valor T y F a lo largo de la suite.

4.2. Casos de prueba

(s_1, s_2, s_3)	triang	Comentario
$(-1, 1, 1)$	4	P_1 : c_1 activa
$(1, -1, 1)$	4	P_1 : c_2 activa
$(1, 1, -1)$	4	P_1 : c_3 activa
$(1, 1, 1)$	3	P_1 falso, equilatero
$(1, 2, 3)$	4	P_6 : d_1 activa
$(2, 3, 4)$	1	P_6 falso, escaleno valido
$(3, 1, 2)$	4	P_6 : d_2 activa
$(1, 3, 2)$	4	P_6 : d_3 activa
$(1, 2, 1)$	4	isosceles imposible ($s_1 + s_3 = s_2$): P_9 falso
$(2, 2, 1)$	2	isosceles via P_8 verdadero
$(1, 1, 2)$	4	P_8/P_{10} : $e_2 = F$, $g_1 = F$
$(2, 1, 2)$	2	isosceles via P_9 verdadero
$(1, 2, 2)$	2	isosceles via P_{10} verdadero
$(2, 1, 1)$	4	P_{10} : g_2 falso

4.3. Pares CACC por predicado

P_1 : $(s_1 \leq 0) \vee (s_2 \leq 0) \vee (s_3 \leq 0)$.

- Mayor c_1 : $(-1, 1, 1)$ vs $(1, 1, 1)$, $c_2, c_3 = F$ en ambos.
- Mayor c_2 : $(1, -1, 1)$ vs $(1, 1, 1)$, $c_1, c_3 = F$ en ambos.
- Mayor c_3 : $(1, 1, -1)$ vs $(1, 1, 1)$, $c_1, c_2 = F$ en ambos.

$P_6: (s_1 + s_2 \leq s_3) \vee \dots$

- Mayor $d_1: (1, 2, 3)$ vs $(2, 3, 4)$.
- Mayor $d_2: (3, 1, 2)$ vs $(2, 3, 4)$.
- Mayor $d_3: (1, 3, 2)$ vs $(2, 3, 4)$.

En cada par, las clausulas d_i no mayores valen F en ambos lados, así que el cambio en P_6 se debe exclusivamente a la clausula mayor.

$P_8: (\mathbf{triOut} = 1) \wedge (s_1 + s_2 > s_3)$.

- Mayor $e_1: (1, 2, 1)$ vs $(2, 2, 1)$. En ambos $e_2 = T$; e_1 cambia de F a T y P_8 pasa de F a T .
- Mayor $e_2: (2, 2, 1)$ vs $(1, 1, 2)$. En ambos $e_1 = T$; e_2 cambia de T a F y P_8 pasa de T a F .

$P_9: (\mathbf{triOut} = 2) \wedge (s_1 + s_3 > s_2)$.

- Mayor $f_1: (1, 1, 2)$ vs $(2, 1, 2)$. En ambos $f_2 = T$; f_1 cambia de F a T .
- Mayor $f_2: (1, 2, 1)$ vs $(2, 1, 2)$. En ambos $f_1 = T$; f_2 cambia de F a T .

$P_{10}: (\mathbf{triOut} = 3) \wedge (s_2 + s_3 > s_1)$.

- Mayor $g_1: (1, 1, 2)$ vs $(1, 2, 2)$. En ambos $g_2 = T$; g_1 cambia.
- Mayor $g_2: (2, 1, 1)$ vs $(1, 2, 2)$. En ambos $g_1 = T$; g_2 cambia.

Predicados unitarios. La misma suite produce, en distintos tests, ambos valores de P_2 , P_3 , P_4 , P_5 y P_7 . Por ejemplo, P_5 vale T en $(1, 2, 3)$ (sin lados iguales) y F en $(1, 1, 1)$.

4.4. Implementacion JUnit

La logica de instrumentacion vive en `TriTypTraceSupport`: para cada entrada genera la secuencia de predicados alcanzados con sus clausulas y, al final, compara la salida *trac-keada* con el valor real devuelto por `TriTyp.triang`. La suite `TriTypExercise4CaccTest` verifica:

- las salidas esperadas de `triang` para los 14 casos;
- los pares CACC para $P_1, P_6, P_8, P_9, P_{10}$ usando un `assertCaccPair` que exige cambio de la clausula mayor, igualdad de las menores y cambio del predicado;
- que la suite alcance ambos valores de P_2, P_3, P_4, P_5 y P_7 .

Ejecucion (desde tp5/assignment-5-rodeghiero):

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
-Djacoco.skip=true test
```

con resultado BUILD SUCCESS y todos los tests verdes.

Conclusiones

A lo largo del practico se aplico la jerarquia de cobertura logica PC, CC, CACC, RACC sobre predicados de complejidad creciente:

- El **Ejercicio 1** muestra que CC y PC son criterios *distintos*: no se subsumen entre si, y se pueden construir suites que satisfacen uno pero no el otro siempre que el predicado admita variabilidad apropiada.
- El **Ejercicio 2** expone la diferencia entre cobertura estructural (aristas) sobre un CFG expandido y cobertura logica (CACC) sobre el predicado original: ambos suben en la jerarquia desde Edge Coverage, pero por caminos distintos, y un test suite valido para uno puede no serlo para el otro.
- El **Ejercicio 3** traslada el analisis a un caso real (**Thermostat**). La diferencia entre CC y PC se ve claramente y la suite CACC obliga a configurar el ambiente para que cada clausula *efectivamente* determine la salida.
- El **Ejercicio 4** ilustra que en un metodo con varios predicados encadenados (**TriTyp.triang**) hay que razonar predicado por predicado y reaprovechar casos para cumplir CACC en cada uno, manteniendo trazabilidad respecto a los valores reales que devuelve el codigo.

La instrumentacion y los asserts vinculan el modelo logico con el comportamiento real del programa, evitando la discordancia entre lo razonado en papel y lo que ocurre en la JVM.